

Sparse Feature Policy Unlearning Mitigates State Hallucination in Vision-Language-Action Models

Anonymous authors

Abstract—Vision-Language-Action (VLA) models have shown strong generalization in robotic manipulation by leveraging rich representations from pretrained vision-language models. However, their deployment in real-world environments remains limited by recurring unreliable behaviors. In this work, we study *state hallucination*, a recurring failure pattern in which a VLA continues acting as if an unrealized robot-object state had been achieved. Our analyses find that state hallucination coincides with weakened attention to task-relevant visual regions, and a mechanistic interpretation via sparse autoencoders reveals that hallucination-associated sparse features are activated when these failures occur. Based on this analysis, we propose *SOUL* (Sparse feature pOlicy UnLearning), which selectively unlearns policy knowledge associated with state hallucination behaviors, where sparse features identified from hallucination failures and successful behaviors serve as explicit forgetting and retention targets, respectively. Experiments across VLA architectures in simulated and real-world environments show that our method substantially reduces hallucinated failures and improves task success without substantially compromising the existing manipulation capabilities. These results suggest that interpretable feature analysis provides a practical basis for selectively modifying undesirable knowledge in robot policies.

I. INTRODUCTION

Vision-language-action models (VLAs) have emerged as a promising approach for bringing foundation models into robot manipulation [1], [2], [3]. By integrating visual and language understanding with action generation, VLAs can translate high-level instructions into continuous robot control. Large-scale pretraining on heterogeneous robot and vision-language data has further enabled generalization across tasks, objects, environments, and robot embodiments [4], [5], [6]. Despite this progress, VLAs still exhibit unreliable behaviors, including unstable or premature actions, long-horizon drift, failures to recover from execution errors, and infeasible actions [7], [8], [9], [10], [11], [12], [13]. This is particularly concerning in physical robot execution, where erroneous actions directly affect the robot and its environment.

Within this broader set of failures, we focus on a recurring failure pattern in which a VLA continues a manipulation sequence based on a robot-object state that has not actually been achieved. We refer to this behavioral pattern as **state hallucination**, because the policy acts as if an unrealized physical state were true. For example, a VLA may close its gripper before reaching an object and subsequently execute transport and place motions as though the object has been successfully grasped, as shown in Fig. 1(a). Such a mismatch can persist across subsequent actions, allowing a local execution error to propagate over multiple stages. This recurring and persistent behavior motivates us to investigate whether policy knowledge associated with state hallucination

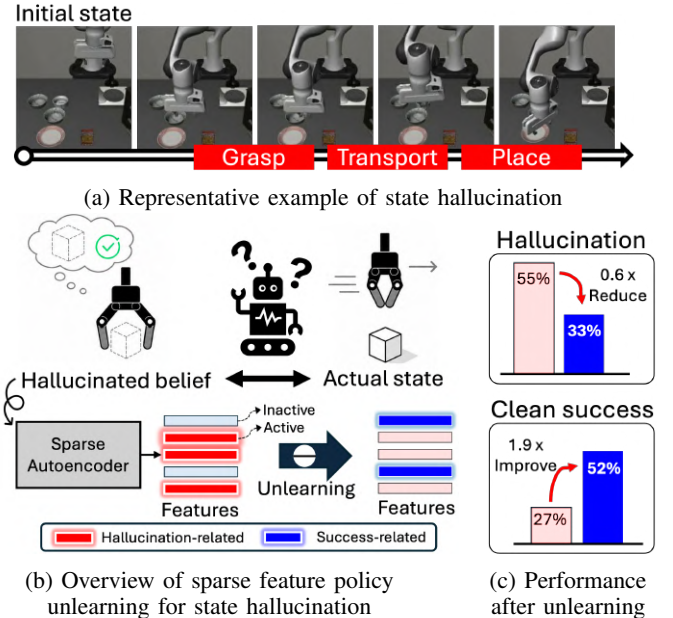


Fig. 1: Sparse feature policy unlearning mitigates hallucinated beliefs in VLAs. (a) Examples of grasp, transport, and place state hallucinations. (b) State hallucination occurs when a VLA mistakes its hallucinated belief for the actual robot-object state and generates actions based on this false belief. During this mismatch, hallucination features become strongly activated, and the proposed unlearning suppresses their activation while reinforcing success features. (c) Our sparse feature policy unlearning reduces hallucination while improving clean (hallucination-free) task success.

can be selectively removed while preserving successful manipulation capabilities.

We first examine how the policy’s internal representations differ between hallucinated and successful behaviors. Such hallucinated rollouts exhibit weakened attention to task-relevant regions, such as the gripper and target object, together with more spatially dispersed attention. Through a mechanistic analysis with sparse autoencoders, we also observe that sparse features exhibit distinct activation patterns during hallucinated and normal executions. To characterize these sparse feature activation patterns, we define a feature association score that contrasts activations across execution outcomes, such as failures with hallucination, failures without hallucination, and successful executions. Based on this score, we categorize sparse features according to whether they are more strongly associated with hallucinated failures or successful executions.

Building on these observations, we propose *Sparse feature policy UnLearning (SOUL)*, which selectively unlearns policy knowledge associated with state hallucination, where sparse features identified from hallucinated failures and successful behaviors serve as explicit forgetting and retention targets, respectively. This feature-level policy unlearning objective suppresses activations of the forgetting targets while enhancing activations of the retention targets, thereby mitigating recurring hallucination behavior while minimizing interference with useful manipulation capabilities. We validate our approach with OpenVLA [2] and $\pi_{0.5}$ [3] on LIBERO-Plus [9], RoboCasa [14], and a real-world Franka robot, achieving an average 21.7%p reduction in hallucination rate and a 22.1%p improvement in task success while preserving 89% of behaviors that were successful before unlearning. These results demonstrate that interpretable feature analysis can reveal internal representations associated with recurring policy failures, providing a practical basis for selectively unlearning such behaviors in embodied foundation models. Our contributions are as follows:

- We characterize state hallucination as a recurring failure pattern in which a VLA continues acting as though an unrealized robot-object state has been achieved.
- We analyze state hallucination through visual attention and sparse features, revealing weakened attention to task-relevant regions and distinct feature patterns associated with state hallucination.
- We introduce a sparse feature policy unlearning method that selectively unlearns recurring state hallucination behavior by suppressing hallucination features while enhancing features that support successful manipulation.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-language-action models have become a major direction for generalist robot manipulation. Prior work has improved generalization through larger pretrained vision-language representations, heterogeneous robotic data, and stronger action generation architectures [1], [2], [3], [4], [5], [6], [15], [16], [17], [18]. Deployment reliability has also been improved through intermediate reasoning [19], [20], failure detection and recovery [12], [21], and test-time sampling or verification [22], [23]. These approaches primarily focus on improving policy outputs during deployment, rather than directly modifying policy knowledge associated with recurring failures. Recent concurrent mechanistic interpretability studies have begun to analyze and steer VLA behavior at inference time by intervening on representations, such as feed-forward activation directions [24] and sparse features [25].

In contrast, we formulate the selective modification of undesirable policy behavior as a policy unlearning problem, where knowledge associated with recurring hallucinated behavior is removed while representations supporting successful execution are preserved. Encoding this modification in the pretrained policy parameters enables persistent behavior correction without repeated intervention at inference time.

B. Machine Unlearning

Machine unlearning aims to make a trained model forget knowledge associated with selected data without retraining the model from scratch [26], [27]. Prior work has studied unlearning mainly in classification tasks, where the influence of selected training data is removed from trained models [28], [29], and in language tasks, where factual knowledge or undesired responses are suppressed [30], [31]. A central challenge is balancing the removal of targeted knowledge with the preservation of useful model capabilities. This trade-off is particularly consequential for robot policies, where modifying one behavior can inadvertently disrupt multiple tightly coupled skills. Motivated by this, we introduce feature-level selective policy unlearning by explicitly identifying features for forgetting and retention to suppress undesirable behavior while preserving useful manipulation capabilities.

C. Sparse Autoencoders

Sparse autoencoders (SAEs) decompose dense model activations into sparse latent representations, enabling individual features to be associated with more localized and interpretable patterns in model behavior [32], [33], [34]. They are trained to reconstruct the original activations under a sparsity constraint, such that only a small subset of latent features is active for each input. A commonly used variant is the Top- k SAE, which imposes this sparsity constraint explicitly by retaining only the k largest latent activations. In VLAs, SAE features have recently been used to interpret and repeatedly steer policy at test time [25]. In contrast, the proposed method identifies recurring internal patterns associated with unreliable behaviors using sparse features and translates these patterns into forgetting targets to guide targeted policy unlearning.

III. ANALYSIS OF STATE HALLUCINATION IN VLAs

A. What Is State Hallucination?

We define **state hallucination** as a behavioral execution pattern in which a VLA continues to generate actions based on a robot-object state that has not actually been achieved. State hallucination can occur at different manipulation stages:

- A *grasp hallucination* occurs when the gripper closes in free space without contacting the target object. Note that grasp failures involving object contact, such as inaccurate localization or slippage, are not considered hallucinations.
- A *transport hallucination* occurs when, after either a grasp failure or a grasp hallucination, the gripper remains closed and continues moving from the grasp location toward the destination while the object remains ungrasped.
- A *place hallucination* occurs when the gripper opens at the destination without holding the target object.

We label these events by tracking ground-truth robot-object states in simulation and observable interaction states in the real world. Based on these criteria, we categorize each rollout as clean success (CS), hallucination failure (HF), or non-hallucination failure (NF). Here, HF denotes failed executions that exhibit at least one state hallucination event, whereas CS and NF denote successful and failed executions without state hallucination, respectively.

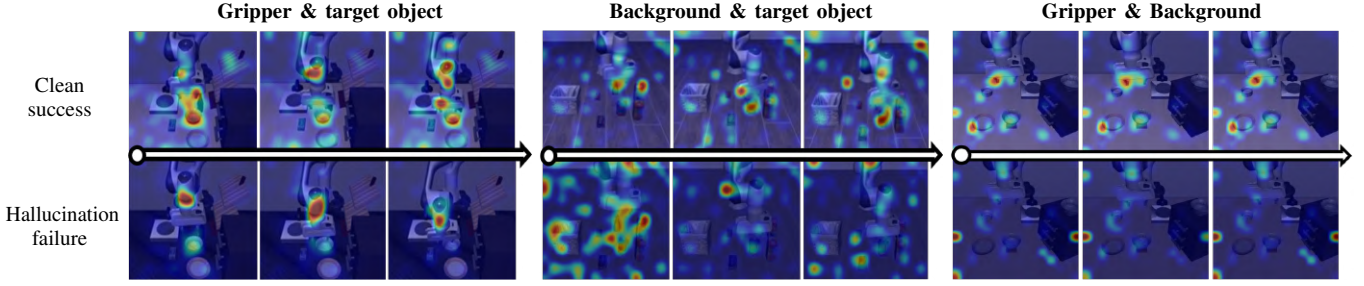


Fig. 2: **State hallucination is associated with distinct attention patterns.** Representative clean success (CS, hallucination-free success) and hallucination failure (HF, failure with hallucination) rollouts under the same task show distinct attention patterns. HF exhibits weaker attention to task-relevant regions (e.g., gripper and target object) and increased attention to the background.

B. Attention Analysis of Hallucinated and Normal Executions

To examine how the policy behaves internally during state hallucination, we compare internal attention patterns between CS and HF executions. At each rollout step, we aggregate the attention from action tokens to visual tokens and project the resulting attention scores onto the input image, revealing which visual regions the policy attends to while generating actions. For visualization, we show representative time steps immediately preceding the corresponding manipulation event, such as grasping. As shown in Fig. 2, CS executions exhibit attention concentrated on task-relevant regions, particularly the target object and gripper. In contrast, HF executions show weaker attention to these regions and increased attention to background regions. These differences suggest that state hallucination is accompanied by a distinct shift in the policy’s visual attention. This motivates us to look beyond attention and examine whether hallucinated executions are also reflected in the internal representations of the policy.

C. Sparse Feature Characterization of State Hallucination

To investigate whether state hallucination is reflected in distinguishable internal representations of VLAs, we analyze policy activations using sparse autoencoders (SAEs) [32]. SAEs decompose dense representations into sparse features, allowing us to characterize how individual feature activations vary across different behaviors, such as clean success, hallucination failure, and non-hallucination failure. Through this characterization, we find that sparse features are selectively associated with state hallucination and successful behaviors.

SAE training. To extract sparse features from policy activations, we collect visual-token activations from the residual streams of transformer layers throughout each rollout together with their execution outcome labels. The visual token activations allow sparse features to be examined with respect to semantic scene regions. For an input sample x , let $x_p \in \mathbb{R}^d$ denote the activation corresponding to visual token p . We flatten the collected activations across rollout steps and visual tokens, treating each x_p as an individual SAE training sample. The encoder maps x_p to a latent representation and retains only the k largest positive activations, yielding a sparse feature vector $z_p(x)$, whose j -th element $z_{p,j}(x)$ denotes the activation of SAE feature j at visual token p . The decoder

reconstructs the original policy activation x_p from $z_p(x)$. After training, we freeze the SAE and use its encoder to extract sparse feature activations for subsequent analysis.

Sparse feature categorization. Motivated by the region-level attention differences observed in Section III-B, we characterize how sparse features are activated across semantic scene regions. We obtain semantic regions from the masks provided by the simulation benchmarks and manually annotate the corresponding regions for real-world observations. For an input sample x , let $\mathcal{R}_r(x)$ denote the set of visual tokens belonging to region r . We define the region-conditioned feature activation as:

$$z_{r,j}(x) = \frac{1}{|\mathcal{R}_r(x)|} \sum_{p \in \mathcal{R}_r(x)} z_{p,j}(x). \quad (1)$$

This quantity measures how strongly feature j is expressed within semantic region r . For rollout e at time step t , we denote the corresponding region-conditioned feature activation by $z_{e,t,r,j}$. Since individual execution events, such as grasp, may occur over limited portions of a rollout, we summarize the temporal activation of each region-feature pair by its maximum over time, $m_{e,r,j} = \max_t z_{e,t,r,j}$. We then average these rollout-level activations within each execution outcome. Let $\mathcal{E}_c$ denote the set of rollouts belonging to outcome $c \in \{\text{CS}, \text{NF}, \text{HF}\}$. The outcome-level activation is computed as:

$$\bar{m}_{c,r,j} = \frac{1}{|\mathcal{E}_c|} \sum_{e \in \mathcal{E}_c} m_{e,r,j}. \quad (2)$$

To examine how individual sparse features are associated with different execution outcomes, we define the following feature association scores:

$$\begin{aligned} s_{r,j}^{\text{hall}} &= \bar{m}_{\text{HF},r,j} - \frac{1}{2} (\bar{m}_{\text{CS},r,j} + \bar{m}_{\text{NF},r,j}), \\ s_{r,j}^{\text{succ}} &= \bar{m}_{\text{CS},r,j} - \frac{1}{2} (\bar{m}_{\text{HF},r,j} + \bar{m}_{\text{NF},r,j}). \end{aligned} \quad (3)$$

Here, larger $s_{r,j}^{\text{hall}}$ and $s_{r,j}^{\text{succ}}$ indicate that feature j at region r is more strongly activated in hallucination failures and clean successes, respectively, relative to the other outcomes. Based on these scores, we categorize sparse features according to their stronger association with hallucination failures or clean successes, and refer to them as hallucination features and success features, respectively.

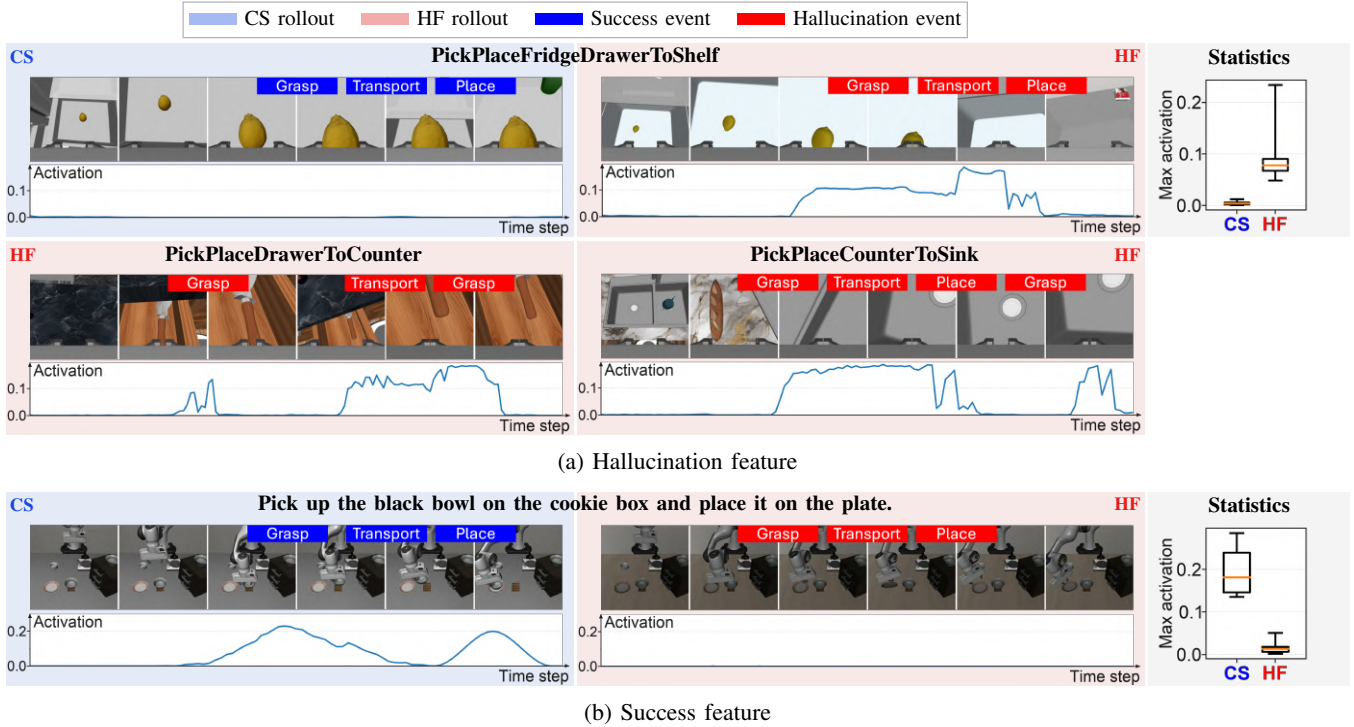


Fig. 3: **Sparse features associated with state hallucination can be identified.** For each feature, we show clean success (CS) and hallucination failure (HF) rollouts with their temporal activations, $z_{e,t,r,j}$, together with the distribution of rollout-level maximum activations, $m_{e,r,j}$. (a) A hallucination feature becomes strongly activated around hallucination events, while remaining weak in CS. (b) A success feature is strongly activated during successful subtask execution or task completion in CS, while remaining weak in HF. The boxplots show consistent activation differences between CS and HF across rollouts.

Sparse features are selectively associated with state hallucination. Fig. 3 visualizes representative hallucination and success features across HF and CS rollouts, showing both their temporal activations and the distributions of their rollout-level activations. For each rollout, the temporal activation $z_{e,t,r,j}$ shows how the sparse feature activation changes over time throughout the rollout. The hallucination feature remains weak during CS rollouts, but becomes strongly activated around state hallucination events in HF rollouts. Its activation is concentrated around hallucinated execution periods rather than remaining uniformly high throughout the rollout, indicating that the feature captures a temporally localized pattern associated with state hallucination. Across HF rollouts spanning different tasks and visual scenes, similar activation patterns repeatedly appear around hallucination events, revealing a consistent feature pattern associated with the behavior. Conversely, the success feature becomes strongly activated during successful subtask execution or task completion in CS rollouts, while remaining weaker in HF.

Beyond these trajectory-level representative rollouts, boxplots on the right summarize the distributions of the rollout-level maximum activation $m_{e,r,j}$ across CS and HF rollouts. Consistent with the temporal activation patterns, the hallucination feature shows higher activation in HF, whereas the success feature shows higher activation in CS. These differences remain consistent across tasks and visual conditions. Together, these results show that state hallucination and successful

execution are associated with distinguishable sparse feature patterns within the VLA representation. More visualizations are provided in Section V and the supplementary materials.

IV. SPARSE FEATURE POLICY UNLEARNING (SOUL)

Building on the sparse feature analysis, we formulate the state hallucination mitigation as a selective policy unlearning problem. Specifically, internal representations associated with state hallucination are suppressed while those supporting successful execution are enhanced. To this end, we target sparse features associated with hallucination failure and clean success as explicit forgetting and retention, respectively. The overall framework is illustrated in Fig. 4.

Feature selection for forgetting and retention. We construct the forget and retain sets using the feature association scores defined in Section III-C. For forgetting, we prioritize features strongly associated with hallucination failures, distinguishable from non-hallucination failures, and weakly associated with clean success. The first hallucination association is captured by $s_{r,j}^{\text{hall}}$, while the other two criteria are combined as $s_{r,j}^{\text{agg}} = s_{r,j}^{\text{sep}} - s_{r,j}^{\text{succ}}$, where $s_{r,j}^{\text{sep}} = \bar{m}_{\text{HF},r,j} - \bar{m}_{\text{NF},r,j}$ measures separation from non-hallucination failures and subtracting $s_{r,j}^{\text{succ}}$ penalizes association with successful execution. We then define the final forget score as $s_{r,j}^{\mathcal{F}} = s_{r,j}^{\text{hall}} + s_{r,j}^{\text{agg}}$ and select the top-ranked features to form the forget set $\mathcal{F}$. For retention, we select features with high $s_{r,j}^{\text{succ}}$ as the retain set $\mathcal{R}$, favoring features strongly associated with

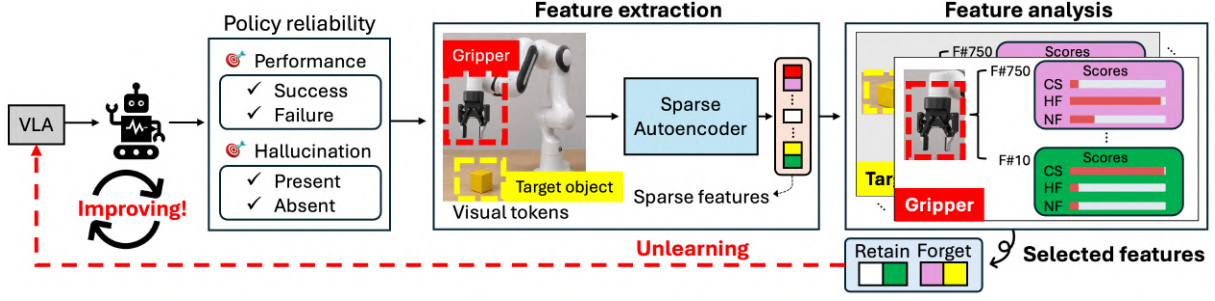


Fig. 4: **Overall framework.** The proposed method provides a framework for improving robot policies by identifying and selectively modifying internal representations associated with undesirable behaviors. It first extracts visual-region-specific sparse features from policy activations and analyzes their activation patterns across execution outcomes, including clean success (CS), hallucination failure (HF), and non-hallucination failure (NF), characterized by task performance and hallucination occurrence. Based on these associations, region-feature pairs are selected as forget or retain targets. The policy is then updated to suppress hallucination-associated features while preserving reliable behavior. In this way, the policy is improved.

clean success. The resulting sets $\mathcal{F}$ and $\mathcal{R}$ define the sparse feature targets to be suppressed and enhanced, respectively, during policy unlearning.

Feature-guided policy unlearning. Given the forget set $\mathcal{F}$ and retain set $\mathcal{R}$, we update the VLA policy using a feature-guided unlearning objective. During unlearning, the SAE encoder remains frozen and is used only to compute sparse feature activations, while the VLA policy parameters θ are optimized. For hallucination failure samples, we define the forgetting objective to penalize activations of the selected forget features as:

$$\mathcal{L}_{\text{forget}} = \mathbb{E}_{x \in \mathcal{D}_{\text{HF}}} \left[\frac{1}{|\mathcal{F}|} \sum_{(r,j) \in \mathcal{F}} (z_{r,j}(x; \theta))^2 \right]. \quad (4)$$

For clean success samples, we define the retention objective to encourage activation of the selected retain features as:

$$\mathcal{L}_{\text{retain}} = -\mathbb{E}_{x \in \mathcal{D}_{\text{CS}}} \left[\frac{1}{|\mathcal{R}|} \sum_{(r,j) \in \mathcal{R}} z_{r,j}(x; \theta) \right]. \quad (5)$$

In summary, the forgetting objective suppresses feature activations associated with state hallucination, while the retention objective reinforces features associated with successful execution, helping preserve useful policy representations during policy unlearning. The final objective jointly balances forgetting and retention as $\mathcal{L}_{\text{unlearn}} = \mathcal{L}_{\text{forget}} + \lambda_r \mathcal{L}_{\text{retain}}$, where λ_r controls the strength of the retention objective.

V. EXPERIMENTS

A. Setup

Evaluation protocol. We conduct experiments with the autoregressive OpenVLA [2] and flow-matching-based $\pi_{0.5}$ [3] in LIBERO-Plus [9], RoboCasa [14], and a real-world setup with a 7-DoF Franka Research 3 robotic arm. LIBERO-Plus provides realistic visual perturbations at multiple severity levels, while RoboCasa provides diverse kitchen environments and everyday manipulation tasks. To provide a controlled setting for evaluating state hallucination while minimizing

confounding effects from task and motion planning, we focus on pick-and-place tasks, a fundamental manipulation primitive underlying many complex tasks, with a structured grasp-transport-place sequence.¹ To our knowledge, no prior work has established an unlearning baseline for robot policies. For a direct comparison from an unlearning perspective, we therefore adapt gradient ascent, a commonly used optimization-based unlearning method [30], [31], as a baseline for VLA policies by maximizing the action prediction loss on hallucination failure samples.

SAE-based feature analysis and unlearning. We use Top- k SAEs [33], [35] to decompose visual-token activations into sparse latent features, whose fixed number of active features provides a consistent basis for comparing activation patterns across executions. The SAE objective combines the standard reconstruction loss with an auxiliary reconstruction loss using the top- k_{aux} inactive features to provide additional learning signals for rarely activated features. We set the SAE latent dimension to 4096, the number of active features to $k = 100$ and $k_{\text{aux}} = 512$. From the sparse feature analysis, we select the top five region-feature pairs associated with hallucination and success as forgetting and retention targets, respectively. For policy unlearning, we initialize the model from the publicly released pretrained policy and optimize LoRA adapters [36] with our unlearning objective while keeping the original model weights frozen. We uniformly sample up to 20 time steps per rollout to reduce bias toward longer trajectories. For feature analysis, we use layers 0, 8, 16, 24, and 31 of the OpenVLA language backbone and layers 0, 5, 11, and 17 of the PaliGemma backbone in $\pi_{0.5}$.

Metrics. To evaluate task performance and state hallucination, we report task-level success metrics and rollout-

¹For LIBERO-Plus, we use the base tasks *put the bowl on the stove*, *pick up the black bowl on the cookie box and place it on the plate*, and *pick up the tomato sauce and place it in the basket* from the goal, spatial, and object suites, respectively, together with their corresponding lighting and background evaluation variants. For RoboCasa, we use *PickPlaceCounterToStandMixer*, *PickPlaceDrawerToCounter*, *PickPlaceCounterToSink*, *PickPlaceCabinetToCounter*, *PickPlaceCounterToCabinet*, and *PickPlaceFridgeDrawerToShelf*, together with their corresponding evaluation configurations.

TABLE I: **Performance comparison of hallucination and task success on benchmarks.** All values are reported as percentages (%). ‘Hall.’ denotes hallucination. ↓ and ↑ indicate that lower and higher values are better, respectively.

Method	Hallucination ↓	Hallucination failure ↓	Overall success ↑	Clean success ↑	Grasp Hall. ↓	Transport Hall. ↓	Place Hall. ↓
LIBERO-Plus with OpenVLA							
Original Policy [2]	70.0	60.0	26.0	16.0	70.0	38.0	16.0
Gradient Ascent [31]	84.0	84.0	0.0	0.0	84.0	62.0	0.0
SOUL (Ours)	44.0	38.0	58.0	52.0	44.0	20.0	6.0
RoboCasa with $\pi_{0.5}$							
Original Policy [3]	40.0	37.1	41.9	39.0	39.0	7.6	1.9
Gradient Ascent [31]	60.0	59.0	4.8	3.8	60.0	0.0	0.0
SOUL (Ours)	22.9	21.0	54.3	52.4	22.9	3.8	0.0

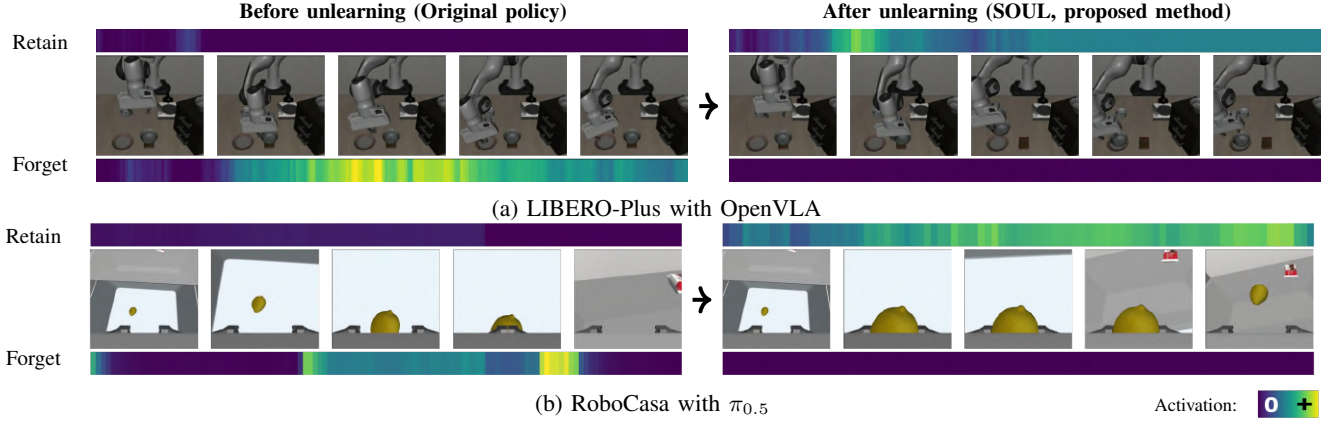


Fig. 5: **Visualizations of rollout behavior and feature activations on benchmarks.** We compare the original policy and the proposed method, showing the temporal activations of selected forget and retain features during execution.

level hallucination metrics. *Overall success* and *clean success* measure the fraction of rollouts that successfully complete the task and the fraction that complete the task without state hallucination, respectively. *Hallucination* and *hallucination failure* measure the fraction of rollouts containing at least one state hallucination event and the fraction that both contain state hallucination and result in task failure, respectively. We further report type-specific hallucination rates for *grasp*, *transport*, and *place*, defined as the fraction of rollouts containing at least one hallucination of each type.

B. Main Results

Quantitative results. To quantitatively evaluate whether our proposed sparse feature policy unlearning (SOUL) improves policy reliability by reducing state hallucination without compromising task success, we compare three policies: the original pretrained policy, gradient-ascent unlearning, and SOUL. As shown in TABLE I, the original policies exhibit average hallucination and hallucination failure rates of 55.0% and 48.6% across evaluation settings, respectively. Gradient-ascent unlearning does not consistently mitigate hallucination and substantially degrades task success, suggesting that directly forgetting hallucination samples can interfere with useful manipulation capabilities. In contrast, SOUL reduces the average hallucination rate from 55.0% to 33.5%, while increasing clean success from 27.5% to 52.2%. Hallucination rates of the grasp, transport, and place stages are also consistently

reduced, showing that the improvement extends across manipulation stages rather than being limited to one stage. These results show that SOUL improves policy reliability by selectively mitigating recurring state hallucination behaviors while preserving useful manipulation capabilities.

Qualitative results. To qualitatively examine how SOUL modifies sparse feature activations, Fig. 5 compares rollout behaviors and the temporal activations of selected forget and retain features before and after applying unlearning across the evaluation settings. Before unlearning, the original policies exhibit state hallucination, with strong forget feature activation emerging around hallucination events while retain feature activation remains weak. After unlearning, the policies successfully complete the tasks without state hallucination, accompanied by suppressed forget feature activation and increased retain feature activation during the corresponding manipulation stages. This pattern is consistently observed for both OpenVLA on LIBERO-Plus and $\pi_{0.5}$ on RoboCasa. These results show that SOUL shifts the selected sparse feature activations in the intended directions, suppressing the activation of hallucination-associated features while increasing that of features associated with clean successful execution.

Real-world evaluation. We further evaluate SOUL on a Franka Research 3 robot using OpenVLA and $\pi_{0.5}$, conducting 50 rollouts across both models under diverse visual conditions. Starting from the publicly released pretrained models, we first fine-tune each model for its target task using

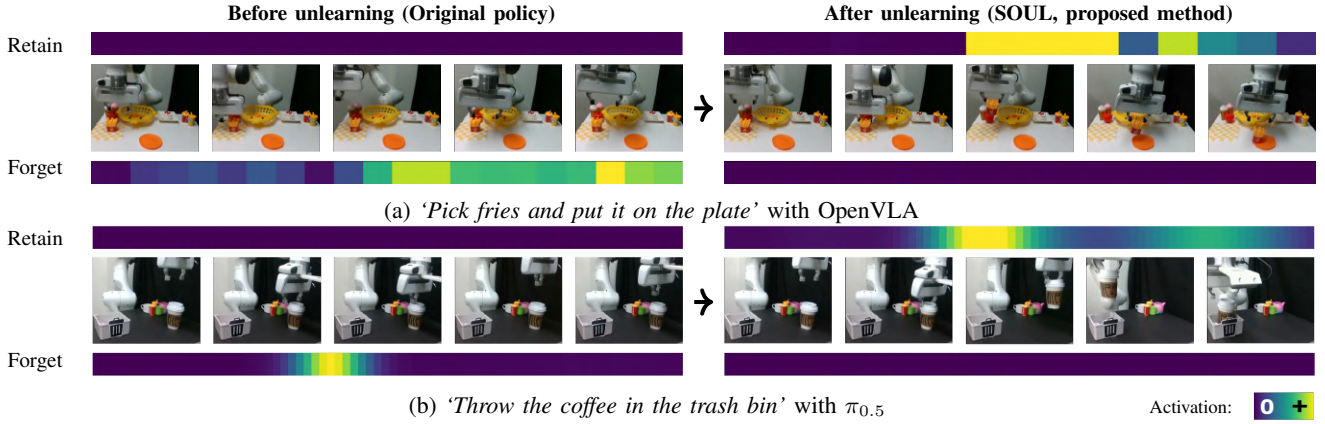


Fig. 6: **Real-world rollouts and selected sparse feature activations before and after applying the proposed unlearning, SOUL.** We compare the original policy and SOUL across representative real-world rollouts spanning both models and tasks.

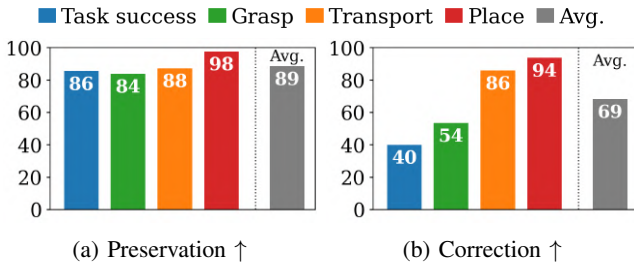


Fig. 7: **Behavior preservation and correction after applying SOUL.** (a) Preservation measures the fraction of originally successful task executions and non-hallucinated events that remain successful and non-hallucinated, respectively, after SOUL. (b) Correction measures the fraction of originally failed task executions and hallucinated events that become successful and non-hallucinated, respectively, after SOUL. Higher values indicate better preservation or correction.

LoRA and then apply SOUL to the task-specific policies. Quantitatively, SOUL increases clean success by an average of 20%p while reducing hallucination failure by 22%p across both settings. As shown in Fig. 6, the original policies exhibit state hallucination, accompanied by strong activation of forget features and weak activation of retain features. After applying SOUL, policies complete the tasks without hallucination, with suppressed forget feature activations and increased retain feature activations. These results show that SOUL remains effective in physical robot execution under diverse real-world visual conditions across both VLA architectures.

C. Analyses

Behavior preservation and correction. To further examine the behavior-level changes underlying the reductions in hallucination and improvements in task success reported in TABLE I, we analyze the unlearning trade-off between preserving desirable behaviors and correcting unreliable behaviors. Fig. 7 reports preservation and correction at both the task and event levels. SOUL achieves an average preservation rate of 89% across originally successful task executions and non-hallucinated events. Meanwhile, it converts 40%

TABLE II: **Comparison of feature selection at different spatial granularities.** ‘Global’ selects features from activations pooled across all visual tokens, ‘Tokenized’ from individual visual-token activations, and ‘Region-guided’ from activations aggregated within each semantic region.

Method	Hall. ↓	Hall. failure ↓	Overall success ↑	Clean success ↑
Global	45.8	41.4	42.7	38.3
Tokenized	41.5	35.5	47.3	41.3
Region-guided	33.5	29.5	56.2	52.2

of originally failed task executions into successful ones and corrects 78% of hallucinated events across the grasp, transport, and place stages. These results demonstrate a favorable unlearning trade-off, substantially correcting undesirable hallucination behaviors while largely preserving behaviors that were already successful or hallucination-free.

Effect of region guidance. To evaluate the effect of spatial grounding on sparse feature selection for policy unlearning, we compare global, tokenized, and region-guided feature selection using OpenVLA and $\pi_{0.5}$ on LIBERO-Plus and RoboCasa, respectively, with results averaged in TABLE II. Global feature selection improves over the original policy, but is less effective than tokenized and region-guided selection, likely because pooling across visual tokens removes spatial information needed to isolate localized hallucination-associated patterns. Tokenized selection performs better by preserving token-level localization, while region-guided selection achieves the lowest hallucination rates and the highest success rates. These results show that preserving spatial structure during feature selection improves policy unlearning, with region guidance achieving the best balance between hallucination reduction and task success.

Contributions of forget score and retain objective. To analyze the roles of the forget score $s_{r,j}^{\mathcal{F}}$ and the retain objective $\mathcal{L}_{\text{retain}}$, we conduct ablations using OpenVLA and $\pi_{0.5}$ on LIBERO-Plus and RoboCasa, respectively, with results averaged in TABLE III. The first row corresponds to the proposed method. Without the retain objective (second row), hallucination reduction is maintained, but clean success

TABLE III: Ablation study of the forget score formulation and retain objective. The aggregated score $s_{r,j}^{agg} = s_{r,j}^{sep} - s_{r,j}^{succ}$ complements $s_{r,j}^{hall}$ for forget feature selection.

$s_{r,j}^{hall}$	$s_{r,j}^{agg}$	$\mathcal{L}_{retain}$	Hall. ↓	Hall. failure ↓	Overall success ↑	Clean success ↑
✓	✓	✓	33.5	29.5	56.2	52.2
✓	✓	✓	36.8	29.9	55.2	48.3
✓	✓	✓	39.9	36.9	50.7	47.7

decreases, showing its importance in preserving successful behavior. Removing the aggregation score from the forget score (third row), both hallucination reduction and task success degrade, showing that incorporating the separation and success criteria enables more selective forgetting target selection. Together, combining score aggregation with the retain objective achieves the best balance, supporting their complementary roles in selective policy unlearning.

VI. CONCLUSION

We have presented a study of selective policy unlearning in VLAs, targeting state hallucination as a recurring undesirable execution behavior. Our analysis showed that state hallucination is accompanied by weakened attention to task-relevant visual regions and distinct region-conditioned sparse feature patterns associated with hallucinated executions. Based on these findings, we proposed *SOUL*, which selectively unlearns policy knowledge associated with state hallucination by suppressing hallucination features while preserving success features. Experiments across VLA architectures in simulated and real-world environments show that *SOUL* reduces state hallucination and improves task success while largely preserving existing manipulation capabilities. These results show that interpretable sparse feature analysis can move beyond diagnosing VLA failures to provide actionable targets for policy unlearning, enabling selective modification of recurring undesirable behaviors in embodied robot policies.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, “Openvla: An open-source vision-language-action model,” in *Conference on Robot Learning*. PMLR, 2025, pp. 2679–2713.
- [3] K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning*, vol. 305. PMLR, 2025, pp. 17–40.
- [4] Open X-Embodiment Collaboration *et al.*, “Open x-embodiment: Robotic learning datasets and RT-X models,” in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 6892–6903.
- [5] NVIDIA *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” in *ArXiv Preprint*, March 2025.
- [6] Gemini Robotics Team *et al.*, “Gemini robotics: Bringing ai into the physical world,” *arXiv preprint arXiv:2503.20020*, 2025.
- [7] Q. Gu *et al.*, “Safe: Multitask failure detection for vision-language-action models,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 40 041–40 076, 2026.
- [8] B. Zhang *et al.*, “Safevla: Towards safety alignment of vision-language-action model via constrained learning,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 153 335–153 373, 2026.
- [9] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.

- [10] X. Zhou, Y. Xu, G. Tie *et al.*, “Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization,” *arXiv preprint arXiv:2510.03827*, 2025.
- [11] Y. Zhang, Y. Qi, and X. Zheng, “Experiences from benchmarking vision-language-action models for robotic manipulation,” *arXiv preprint arXiv:2511.11298*, 2025.
- [12] Z. Lin, J. Duan, H. Fang *et al.*, “Failsafe: Reasoning and recovery from failures in vision-language-action models,” *arXiv preprint arXiv:2510.01642*, 2025.
- [13] H. Soh and E. Lim, “Action hallucination in generative visual-language-action models,” *arXiv preprint arXiv:2602.06339*, 2026.
- [14] X. Zhou *et al.*, “RoboCasa: Large-scale simulation of household tasks for generalist robots,” in *Robotics: Science and Systems XX*, 2024.
- [15] D. Driess *et al.*, “Palm-e: an embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 8469–8488.
- [16] D. Ghosh *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [17] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [18] M. Shukor *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [19] Q. Zhao *et al.*, “Cot-vla: Visual chain-of-thought reasoning for vision-language-action models,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 1702–1713.
- [20] F. Lin, R. Nai, Y. Hu, J. You *et al.*, “OnetwoVLA: A unified vision-language-action model with adaptive reasoning,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [21] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen, “Self-correcting vla: Online action refinement via sparse world imagination,” *arXiv preprint arXiv:2602.21633*, 2026.
- [22] J. Kwok *et al.*, “Robomoney: Scaling test-time sampling and verification for vision-language-action models,” in *Conference on Robot Learning*. PMLR, 2025, pp. 3200–3217.
- [23] S. Jang, D. Kim, C. Kim, Y. Kim, and J. Shin, “Verifier-free test-time sampling for vision-language-action models,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [24] B. Häon, K. C. Stocking, I. Chuang, and C. Tomlin, “Mechanistic interpretability for steering vision-language-action models,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2743–2762.
- [25] M. A. Khan, N. Boskov, F. M. Anwar, and M. A. Khan, “Controlling vision-language-action policies through sparse latent directions,” in *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025.
- [26] Y. Cao and J. Yang, “Towards making systems forget with machine unlearning,” in *IEEE symposium on security and privacy*, 2015.
- [27] T. T. Nguyen, T. T. Huynh, Z. Ren, P. L. Nguyen, A. W.-C. Liew, H. Yin *et al.*, “A survey of machine unlearning,” *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, pp. 1–46, 2025.
- [28] A. Golatkar, A. Achille, and S. Soatto, “Eternal sunshine of the spotless net: Selective forgetting in deep networks,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2020, pp. 9301–9309.
- [29] A. Thudi *et al.*, “Unrolling SGD: Understanding factors influencing machine unlearning,” in *IEEE European symposium on security and privacy*, 2022.
- [30] Y. Yao and X. Xu, “Large language model unlearning,” *Advances in Neural Information Processing Systems*, pp. 105 425–105 475, 2024.
- [31] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, “Tofu: A task of fictitious unlearning for llms,” *arXiv preprint arXiv:2401.06121*, 2024.
- [32] R. Huben *et al.*, “Sparse autoencoders find highly interpretable features in language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [33] L. Gao *et al.*, “Scaling and evaluating sparse autoencoders,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 26 721–26 754.
- [34] B. Bussmann, P. Leask, and N. Nanda, “Batchtopk sparse autoencoders,” *arXiv preprint arXiv:2412.06410*, 2024.
- [35] A. Makhzani and B. Frey, “K-sparse autoencoders,” *arXiv preprint arXiv:1312.5663*, 2013.
- [36] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.